Università
Ca'Foscari
Venezia

Local Search

FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

Andrea Torsello

Dip. Sc. Ambientali, Informatica, Statistica
Università Ca'Foscari Venezia

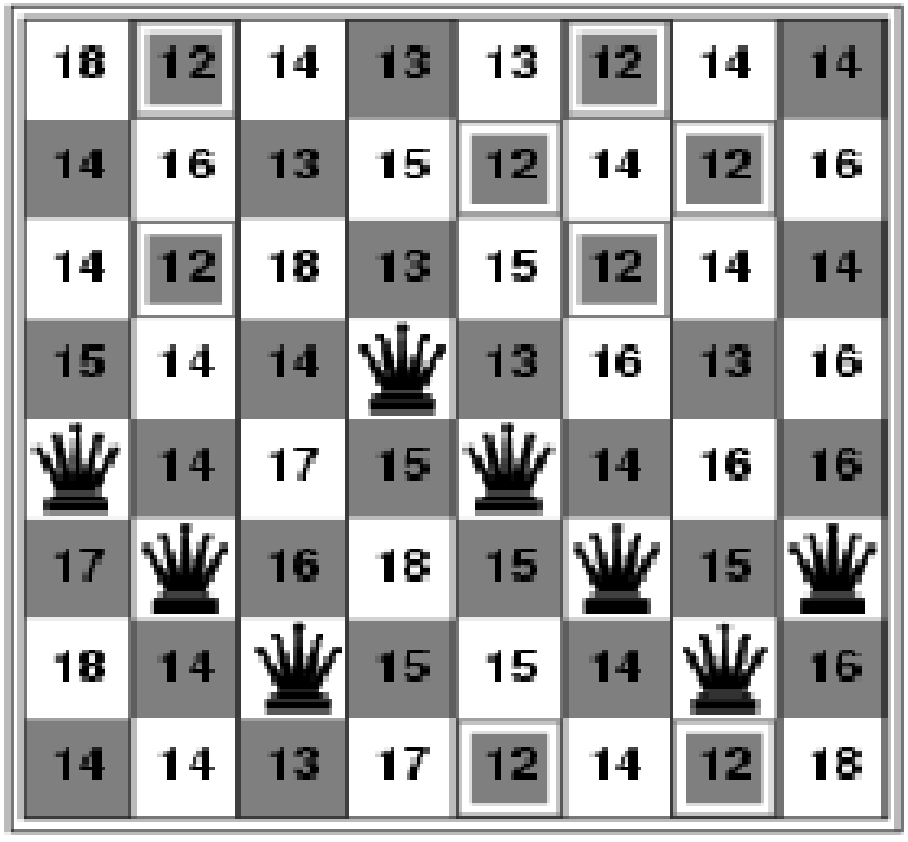
Constraint Satisfaction as Optimization

A approach to contraint satisfaction problems is to turn them into an optimization problem

Define an **objective fuction** that is

- positive
- 0 if and only if all constrained are satisfied

Example: N-Queens



h = number of pairs of queens that are attacking each other, either directly or indirectly

h = 17 for the above state

Local Search

Search from an optimum by perforing (local) minimization or maximization

Move from one state to an adjacent one in an attempt to derease (increase) the objective function

Requires:

- notion of adjacent states;
- strategy to decide on which of the available states to move

Hill-Climbing

Simplest approach:

- always move to the adjacent state that minimizes (maximizes) the objective function;
- terminate if adjacent states do not improve on current state.

Discrete version of gradient descent (ascent)

One problem is the presence of Local minima (maxima)

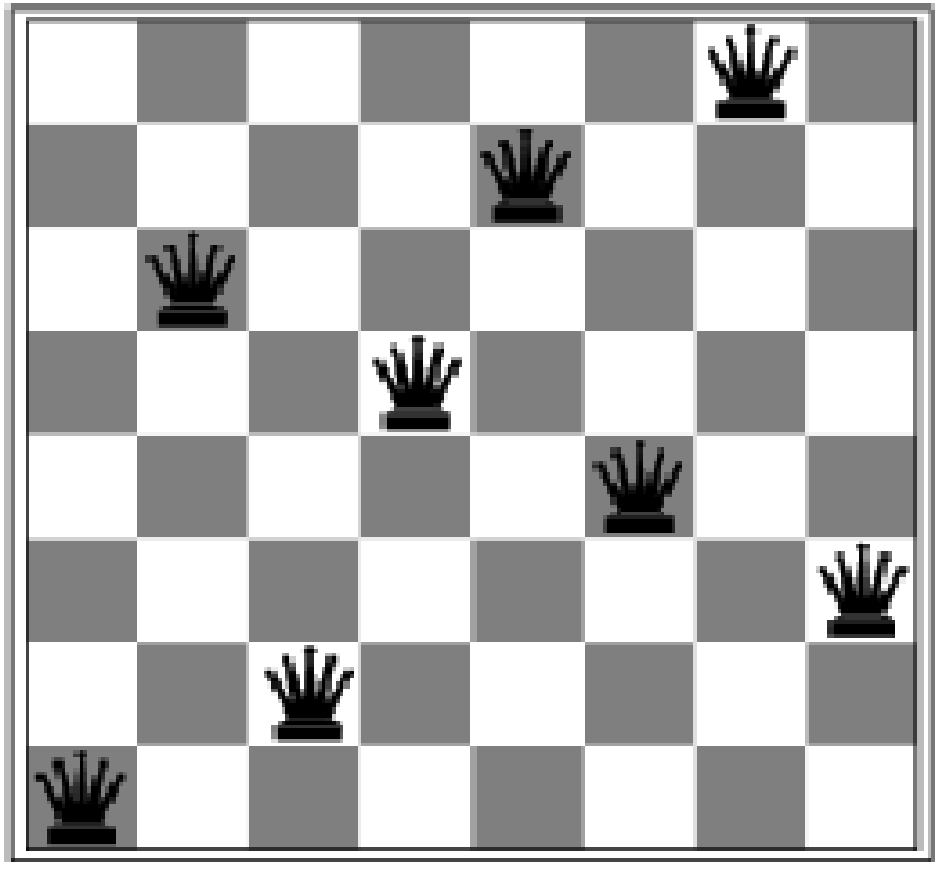
Hill-Climbing

Example (Hill-Climbing)

```
def hill_climbing(f,initial):
  current=initial
  while True:
    best=current
    for i in neighbours(current):
      if f(i) > f(best):
        best=i
    if best==current:
      return current
    current=best
```

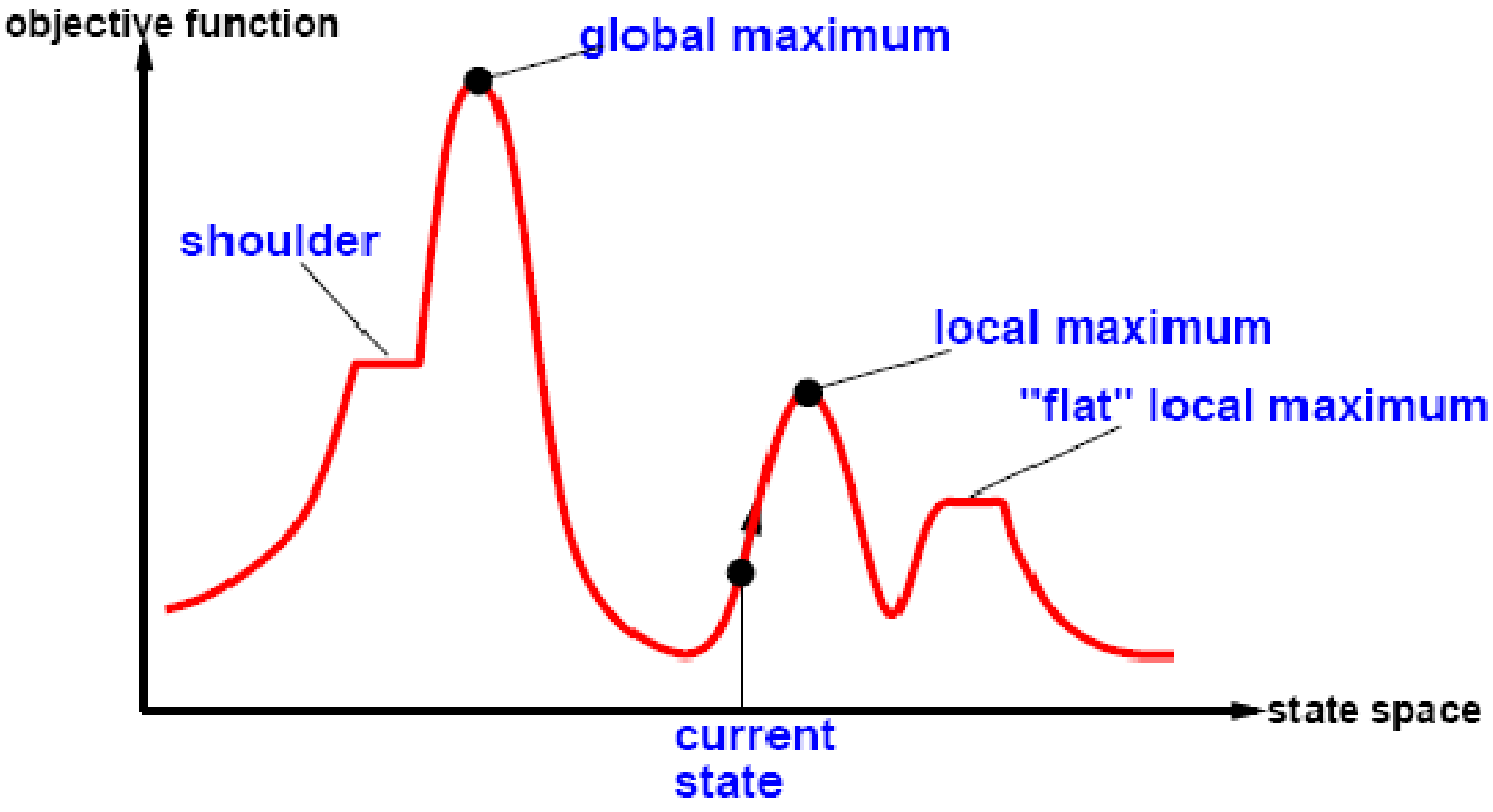
Università
Ca'Foscari
Venezia

Example: N-Queens Local Minimum



A local minimum with $h = 1$

Optimization Landscape



Global Optimization

We need algorithms that can escape local optima and (hopefully) converge to the global optimum

Several strategies:

- restart with several initial condition;
- allow *on occasion* to make increasing (decreasing) moves;
- introduce random jumps away from the neighborhood;
- etc.

Simulated annealing

Simulated annealing is a probabilistic technique for approximating the global minimum (maximum) of a given function f

The name of the algorithm comes from annealing in metallurgy, a technique involving heating and controlled cooling of a material to alter its physical properties

The simulated annealing heuristic considers some neighboring state s^* of the current state s, and probabilistically decides between moving the system to state s^* or staying in-state s

Simulated Annealing

The probability of making the transition from the current state s to a candidate new state s^* is specified by an acceptance probability function $P(e, e^*, T)$, that depends on

- the energies (objective values) $e = f(s)$ and $e^* = f(s^*)$ of the two states
- a global time-varying parameter T called the temperature

When T tends to zero, the probability$P(e, e^*, T)$ must tend to zero if $e^* > e$ and to a positive value otherwise

For sufficiently small values of T, the system will then increasingly favor moves that go "downhill" (i.e., to lower energy values), and avoid those that go "uphill."

Simulated Annealing

Example (Simulated Annealing)

```
def simulated_annealing(f, initial):
  current=initial
  for T in schedule:
    if T==0:
      return current
    next = random_select(neighbor(current))
    de = f(next)-f(current)
    if de < 0:
      current=next
    else:
      if random() < exp(-de/T):
        current=next
```

Università
Ca'Foscari
Venezia

Properties of Simulated Annealing

If T decreases slowly enough, then simulated annealing search will find a global optimum with probability approaching 1

Università
Ca'Foscari
Venezia

Local beam search

Keep track of k states rather than just one

Start with k randomly generated states

At each iteration, all the successors of all k states are generated

If any one is a goal state, stop; else select the k best successors from the complete list and repeat.

Università
Ca'Foscari
Venezia

Genetic algorithms

A successor state is generated by combining two parent states

Start with k randomly generated states (population)

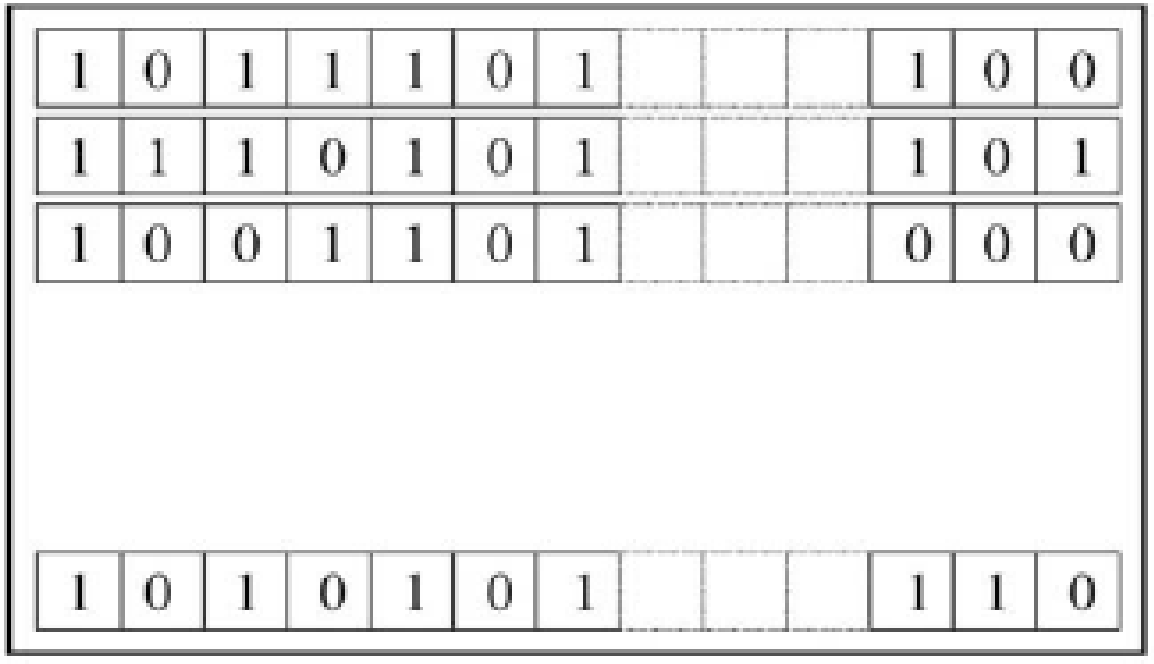
A state is represented as a string over a finite alphabet (often a string of 0s and 1s)

Evaluation function (fitness function). Higher values for better states.

Produce the next generation of states by selection, crossover, and mutation

Encoding

Operate on a population P of n strings (individuals or genotypes)



Genotype 1
Genotype 2
Genotype 3

⋮

Genotype *n*

Selection

Darwinian selection holds that "stronger" individuals have greater probability of surviving and of reproducing

In the context of Genetic Algorithms, stronger individuals are those with higher fitness

- Individuals with higher fitness are more likely to be selected for reproduction
- Individuals selected for reproduction are placed in the mating pool
- When the mating pool is filled with n copies of the individuals, the descendants are created applying the genetic operators (mutation and crossover)

Selection

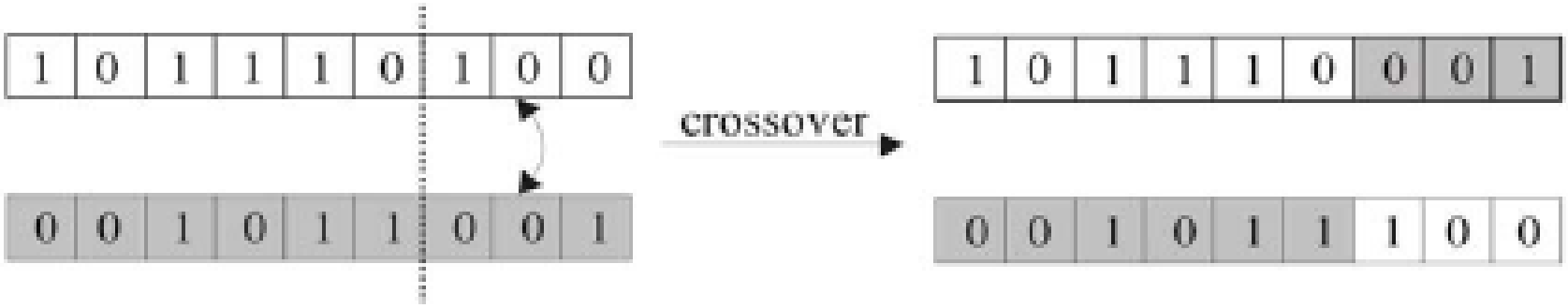
Roulette wheel selection

let f_i be the fitness value of individual i, the probability that i is selected for reproduction is

$$p_i = \frac{f_i}{\sum_k f_k}$$

Crossover

Select two parents and a crossover point
The right portion of the genotypes are swapped with probability pc, generating two descendants

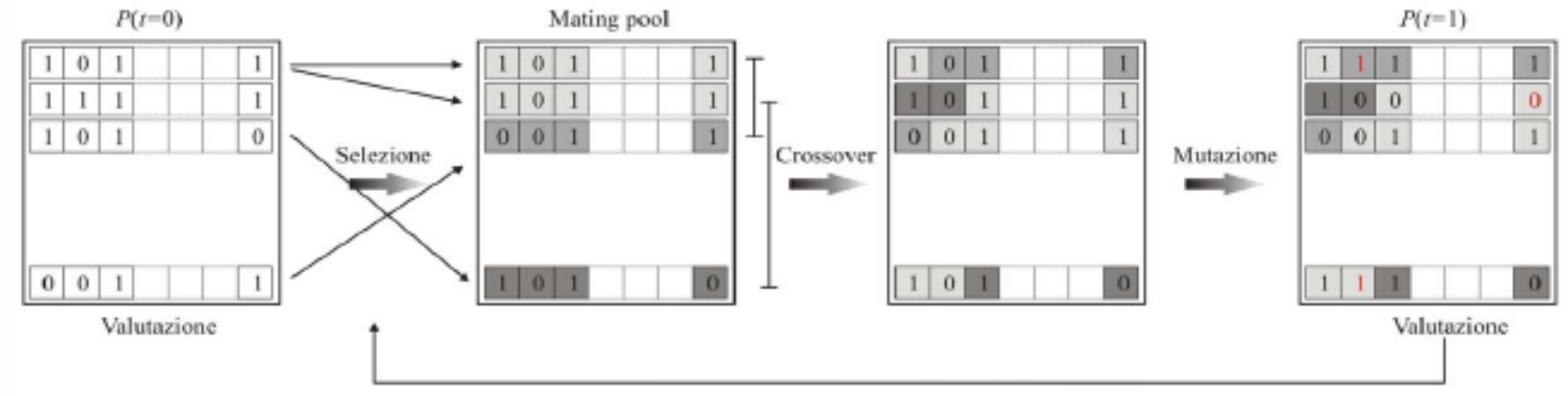


Mutation

With (low) probability p_m one location of the genotype is altered

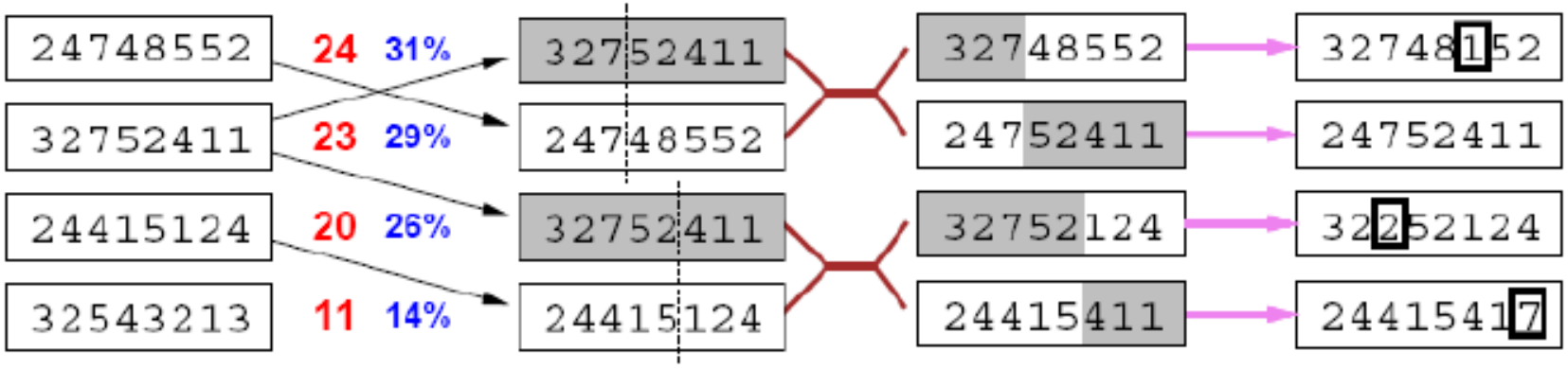
Università
Ca'Foscari
Venezia

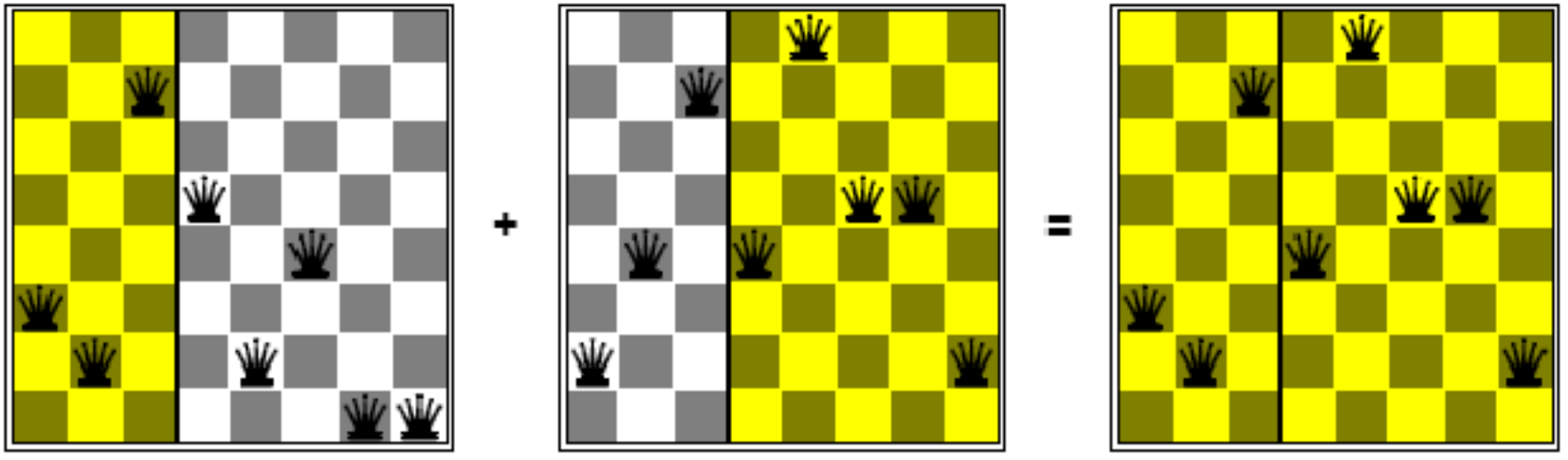
Simple Genetic Algorithm



Università
Ca'Foscari
Venezia

Example: N-Queen





Search in continuous space

It is sometimes useful to convert the search problem into one on a **continuous domain**

Process called **relaxation**

- The discreteness of the domain is *relaxed* to a continuous settin where we can use calculus and continuous optimization techniques

A classic approach for Constraint Satisfaction problem is to convert the state

- from a *crisp* assignment of a variable to a value in its domain
- to a **probabilistic** assigment (*i.e.*, the state is a set of **distributions** over the domains)

Optimization problems

The general mathematical programming problem can be stated as

$$\begin{aligned} \text{minimize} \quad & f(\mathrm{x}) \\ \text{subject to} \quad & h_i(\mathrm{x}) = 0, \qquad i = 1, 2, \ldots, m \\ & g_j(\mathrm{x}) \leq 0, \qquad j = 1, 2, \ldots, r \\ & \mathrm{x} \in \Omega\,. \end{aligned}$$

In this formulation

- $\mathrm{x} = (x_1, x_2, \ldots, x_n)$ is an n-dimensional vector of unknowns
- f, h_i and g_i are real-valued functions of x
- Ω is a subset of the n-dimensional real space
- the function f is the objective function of the problem
- the equations, inequalities and set restrictions are constraints

Optimization problems

Assume the generic optimization function in the form

$$\begin{aligned} \text{minimize} \quad & f(x) \\ \text{subject to} \quad & x \in \Omega\,, \end{aligned}$$

where f is the objective function and $\Omega \in \mathbb{R}^n$ is the feasible set

Definition

A point x^* is said to be *relative minimum point* or a *local minimum point* pf f over Ω if there is an $\epsilon > 0$ such that $f(x) \geq f(x^*)$ for all $x \in \Omega$ within a distance ϵ of x^* (*i.e.*, $|x - x^*| < \epsilon$). If $f(x) > f(x^*)$ for all $x \in \Omega, x \neq x^*$, within a distance ϵ of x^*, then x^* is said to be a *strict relative minimum point* of f over Ω.

Optimization problems

Assume the generic optimization function in the form

$$\begin{aligned} \text{minimize} \quad & f(x) \\ \text{subject to} \quad & x \in \Omega\,, \end{aligned}$$

where f is the objective function and $\Omega \in \mathbb{R}^n$ is the feasible set

Definition

A point $x^* \in \Omega$ is said to be a *global minimum point* of f over Ω if $f(x) \geq f(x^*)$ for all $x \in \Omega$. If $f(x) > f(x^*)$ for all $x \in \Omega, x \neq x^*$, then x^* is said to be a *strict global minimum point* of f over Ω.

Types of optimization problems

- Linear versus nonlinear optimizaton
- Unconstrained versus constrained optimization

Linear Programming

$$\begin{aligned} \text{minimize} \quad & c^T x \\ \text{subject to} \quad & Ax = b \quad \text{and} \quad x \geq 0\,. \end{aligned} \tag{1}$$

Algrithms:

- simplex method
- interior-point method

Gradient Descent

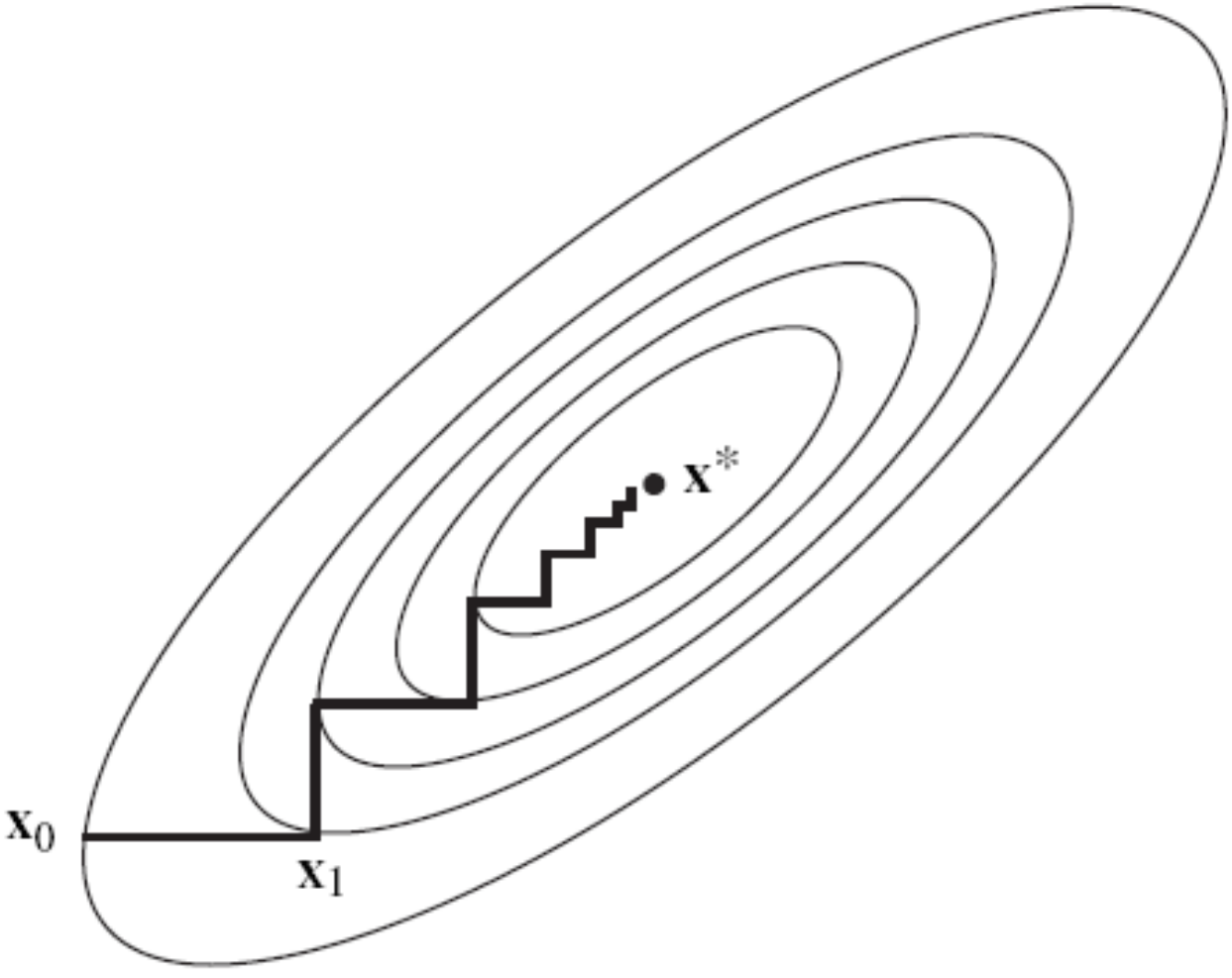
Let f have continuous first partial derivatives on E^n. We will frequently have need for the gradient vector of f and therefore we introduce some simplifying notation. The gradient $\nabla f(\mathbf{x})$ is, according to our conventions, defined as a n-dimensional *row* vector. For convenience we define the n-dimensional *column* vector $\mathbf{g}(\mathbf{x}) = \nabla f(\mathbf{x})^T$. When there is no chance for ambiguity, we sometimes suppress the argument $\mathbf{x}$ and, for example, write $\mathbf{g}_k$ for $\mathbf{g}(\mathbf{x}_k) = \nabla f(\mathbf{x}_k)^T$.

The method of steepest descent is defined by the iterative algorithm

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha_k \mathbf{g}_k,$$

where α_k is a nonnegative scalar minimizing $f(\mathbf{x}_k - \alpha \mathbf{g}_k)$. In words, from the point $\mathbf{x}_k$ we search along the direction of the negative gradient $-\mathbf{g}_k$ to a minimum point on this line; this minimum point is taken to be $\mathbf{x}_{k+1}$.

Gradient Descent



Newton's method

The idea behind Newton's method is that the function f being minimized is approximated locally by a quadratic function, and this approximate function is minimized exactly. Thus near $\mathbf{x}_k$ we can approximate f by the truncated Taylor series

$$f(\mathbf{x}) \simeq f(\mathbf{x}_k) + \nabla f(\mathbf{x}_k)(\mathbf{x} - \mathbf{x}_k) + \tfrac{1}{2}(\mathbf{x} - \mathbf{x}_k)^T \mathbf{F}(\mathbf{x}_k)(\mathbf{x} - \mathbf{x}_k).$$

The right-hand side is minimized at

$$\mathbf{x}_{k+1} = \mathbf{x}_k - [\mathbf{F}(\mathbf{x}_k)]^{-1} \nabla f(\mathbf{x}_k)^T,$$

and this equation is the pure form of Newton's method.

Constrained Optimization

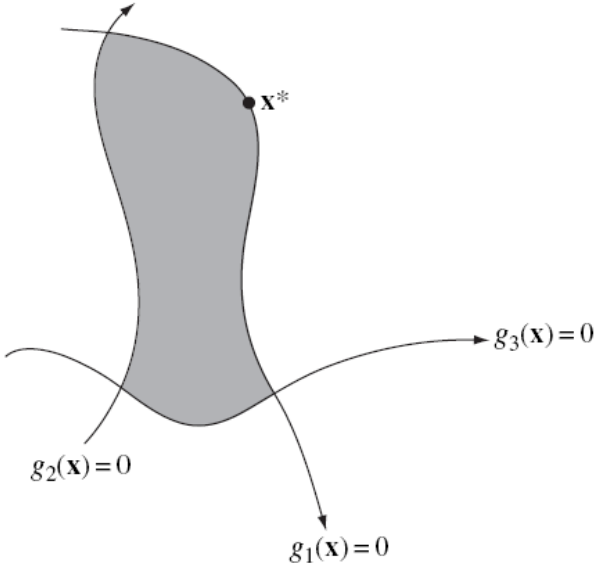
$$\begin{array}{ll} \text{minimize} & f(\mathbf{x}) \\ \text{subject to} & \mathbf{h}(\mathbf{x}) = \mathbf{0},\ \mathbf{g}(\mathbf{x}) \leqslant \mathbf{0} \\ & \mathbf{x} \in \Omega. \end{array}$$

The constraints $\mathbf{h}(\mathbf{x}) = \mathbf{0}, \mathbf{g}(\mathbf{x}) \leqslant \mathbf{0}$ are referred to as *functional constraints*, while the constraint $\mathbf{x} \in \Omega$ is a *set constraint*. As before we continue to de-emphasize the set constraint, assuming in most cases that either Ω is the whole space E^n or that the solution to (2) is in the interior of Ω. A point $\mathbf{x} \in \Omega$ that satisfies all the functional constraints is said to be *feasible*.

Università
Ca'Foscari
Venezia

Constrained Optimization

$$
\begin{aligned}
\text{minimize} \quad & f(x) \\
\text{subject to} \quad & h_1(x) = 0 \quad g_1(x) \leq 0 \\
& h_2(x) = 0 \quad g_2(x) \leq 0 \\
& \vdots \\
& h_m(x) = 0 \quad g_p(x) \leq 0 \\
& x \in \Omega\,,
\end{aligned}
$$



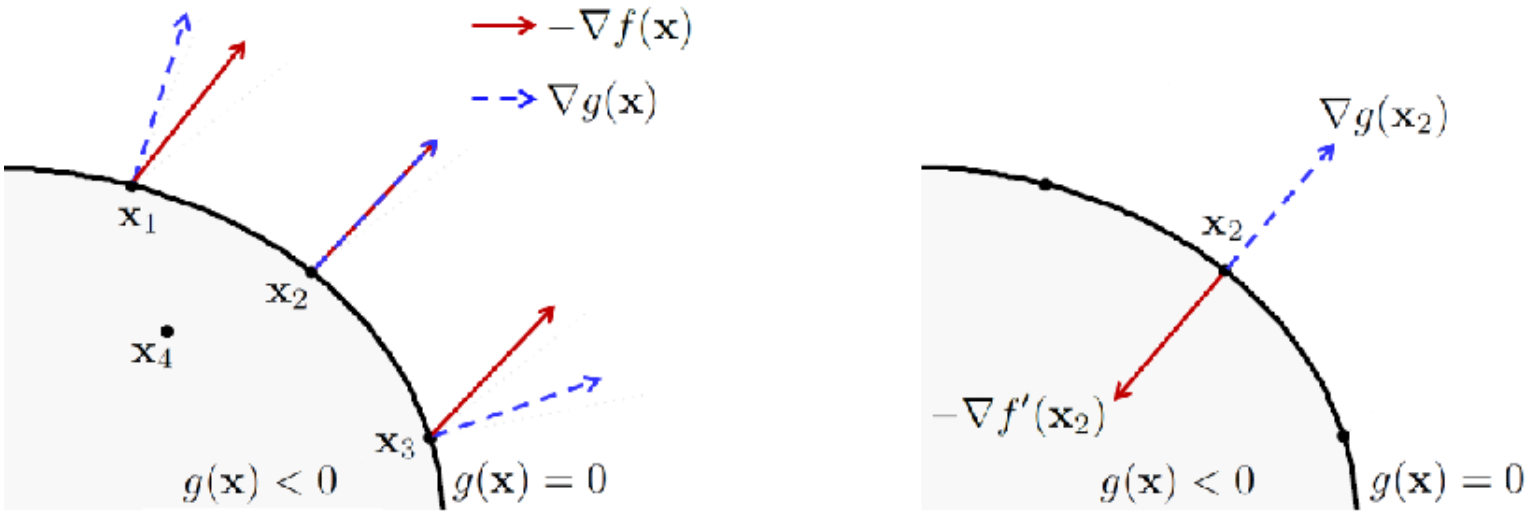
KKT Conditions

$$\begin{aligned} \text{minimize} \quad & f(x) \\ \text{subject to} \quad & h(x) = 0 \\ & g(x) \leq 0 \end{aligned}$$

Let x^* be a local extremum point of f subject to the constraints $h(x) = 0$ (equality contraints) and $g(x) \leq 0$ (inequality constraints). Assume futher that x^* is a regular point of these constraints. Then there is a $\lambda \in \mathbb{R}^m$ and a $\mu \geq 0 \in \mathbb{R}^p$ such that

$$\nabla f(x^*) + \lambda^T \nabla h(x^*) + \mu^T \nabla g(x^*) = 0$$

KKT Geometric Interpretation



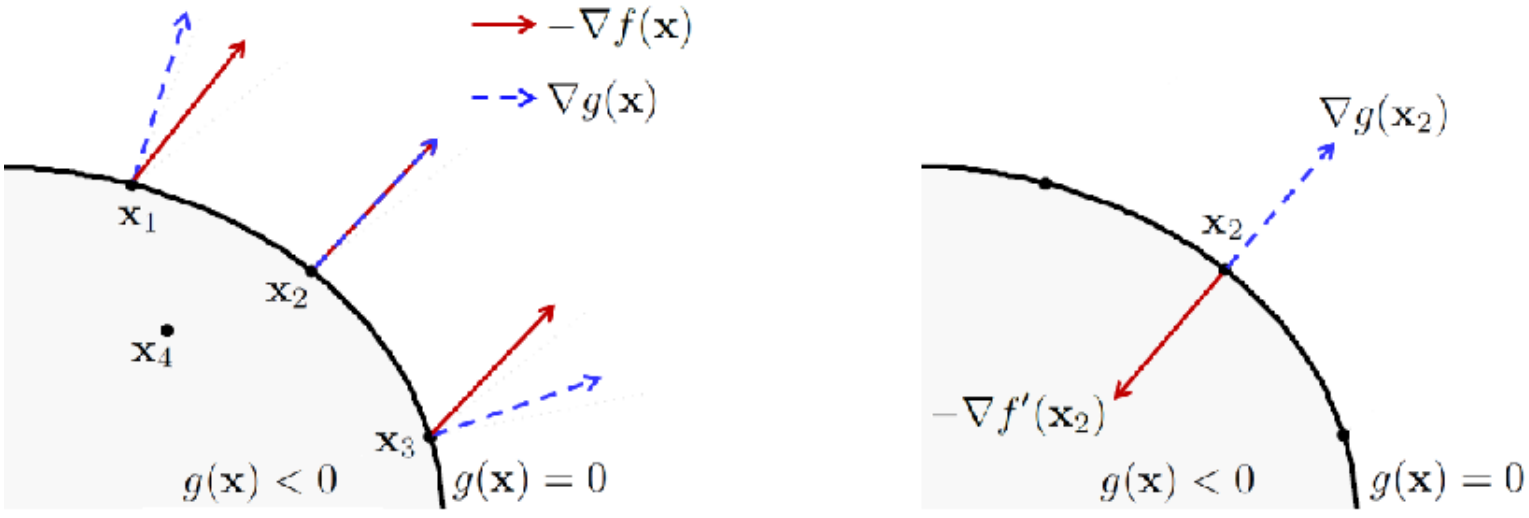
The gradient of the constraint $\nabla g(\mathrm{x})$ is **orthogonal** to the boundary of the feasible set

If the gradient of the objective function $\nabla f(\mathrm{x})$ is not aligned with $\nabla g(\mathrm{x})$, then there is a growth direction parallel to the boundary of the feasible set.

- $\nabla f(\mathrm{x})$ must be aligned with $\nabla g(\mathrm{x})$

Università
Ca'Foscari
Venezia

KKT Geometric Interpretation

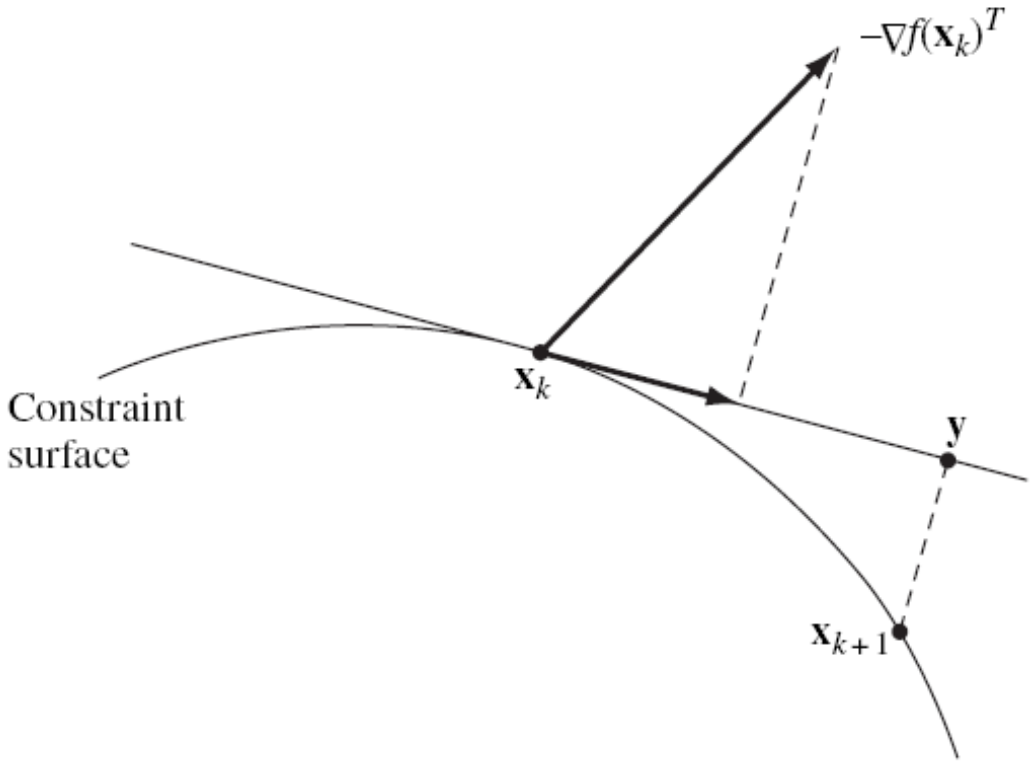


For inequality constraints, if the decreasing direction $-\nabla f(\mathrm{x})$ points in the direction of negative $g()\mathrm{x})$, *i.e.*, opposite to $\nabla g(\mathrm{x})$, then the direction is fesible, otherwse not.

- $\nabla f(\mathrm{x})$ must be a positive multiple of $\nabla g(\mathrm{x})$ or 0.

The Gradient Projection Method

The gradient projection method is motivated by the ordinary method of gradient descent for unconstrained problems. The negative gradient is projected onto the working surface in order to define the direction of movement



Penalty Methods

Penalty methods approximate a constrained problem by an unconstrained problem that assigns high cost to points that are far from the feasible region. To this end we transform the problem

$$\begin{aligned} \text{minimize} \quad & f(\mathrm{x}) \\ \text{subject to} \quad & \mathrm{x} \in \Omega \end{aligned} \tag{2}$$

into the relaxed problem

$$\text{minimize} f(\mathrm{x}) + cP(\mathrm{x})$$

where c is a positive constant and P is a function satisfying

- P is continuous
- $P(\mathrm{x}) \geq 0$ for all $\mathrm{x} \in \mathbb{R}^n$
- $P(\mathrm{x}) = 0$ if and only if $\mathrm{x} \in \Omega$

Penalty Methods

As the approximation is made more exact (by letting the parameter c tend to infinity) the solution of the unconstrained penalty problem approaches the solution to the original constrained problem from outside the active constraints.

Example 1. Suppose S is defined by a number of inequality constraints:

$$S = \{\mathbf{x} : g_i(\mathbf{x}) \leqslant 0, \quad i = 1, 2, \ldots, p\}.$$

A very useful penalty function in this case is

$$P(\mathbf{x}) = \tfrac{1}{2} \sum_{i=1}^{P} (\max\,[0, g_i(\mathbf{x})])^2.$$

Lagrange Multipliers

The method of Lagrange multipliers is a strategy for finding the local maxima and minima of a function subject to equality constraints

$$\begin{aligned} \text{minimize} \quad & f(x) \\ \text{subject to} \quad & h(x) = 0 \end{aligned}$$

Define the *Lagrangian function* as

$$\mathcal{L}(x, \lambda) = f(x) + \lambda^T h(x)$$

The gradient of the Lagrangian function is

$$\nabla \mathcal{L} = \begin{bmatrix} \nabla_x \mathcal{L} \\ \nabla_\lambda \mathcal{L} \end{bmatrix} = \begin{bmatrix} \nabla f(x) + \lambda^T \nabla h(x) \\ g(x) \end{bmatrix}$$

Lagrange Multipliers

$\nabla \mathcal{L} = 0$ is equivalent to the combination of the constaints and the KKT conditions

- we transform a constrained problem into and unconstrained search for the zeroes of the gradient of the Lagrange function

Unfortunately this is not an optimization problem as thie zero is actually a **saddle point**

- a minimum with respect to x
- a maximum with respect to λ

Lagrange Multipliers

Let $x^*(\lambda)$ be the value of x that minimizes $\mathcal{L}(x, \lambda)$ for a given λ

We define the reduced Lagrangian function

$$\mathcal{L}^*(\lambda) = \mathcal{L}(x^*(\lambda), \lambda)$$

The **maximum** λ^* of this function solves the original optimization problem, *i.e.* $x^*(\lambda^*)$ is a soution of the original constrained optimization problem

- we turned a contrained problem into an unconstrained one.

A Special Case...

We will consider the following optimization problem:

$$\begin{aligned} \min_x \ (\max_x) \quad & x^T A x \\ \text{subject to} \quad & x^T x = 1 \end{aligned}$$

with A a symmetric matrix, and its generalizations

Rayleigh-Ritz Theorem

Theorem (Rayleigh-Ritz)

Let A be a symmetric $n \times n$ matrix with eignevectors $\lambda_1 \leq \lambda_2 \leq \ldots \leq \lambda_n$, and let $x \in \mathbb{R}^n$ be a vector satisfying $x^T x = 1$. The quantity $x^T Ax$ satisfies

$$\lambda_1 \leq x^T Ax \leq \lambda_n$$

with the extremes reached for $x = \phi_1$ and $x = \phi_n$ respectively, where ϕ_1 is an eigenvalue associated with λ_1 and ϕ_n is an eigenvalue associated with λ_n.

Rayleigh-Ritz Theorem

Proof

Express x in terms of the eignevalues $\phi_1, \ldots, \phi_n$:

$$x = \sum_{i=1}^{n} \alpha_i \phi_i$$

for some α_i satisfying $\sum_{i=1}^{n} \alpha_i^2 = 1$ so that $x^T x = 1$

Then:

$$\begin{aligned} x^T A x &= \left(\sum_{i=1}^{n} \alpha_i \phi_i\right)^T A \left(\sum_{i=1}^{n} \alpha_i \phi_i\right) = \left(\sum_{i=1}^{n} \alpha_i \phi_i\right)^T \left(\sum_{i=1}^{n} \alpha_i A \phi_i\right) \\ &= \left(\sum_{i=1}^{n} \alpha_i \phi_i\right)^T \left(\sum_{i=1}^{n} \alpha_i \lambda_i \phi_i\right) = \sum_{i=1}^{n} \alpha_i^2 \lambda_i \end{aligned}$$

Rayleigh-Ritz Theorem

Let $\beta_i = \alpha_i^2$, we have the equivalent optimization problem

$$\min_{\beta} (\max_{\beta}) \quad \sum_{i=1}^{n} \beta_i \lambda_i$$

$$\text{subject to} \quad \sum_{i=1}^{n} \beta_i = 1$$

$$\beta_i \geq 0$$

Clearly this obtains the minimum (maximum) when $\beta_i = 1$ for the i for which λ_i is minimum (maximum), and 0 for all the other is.

...and the value of $\sum_{i=1}^{n} \beta_i \lambda_i$ is such minimum (maximum) λ_i

The fact that $x^T Ax$ reaces those values on the eigenvectors is pretty straightforward

□

Generalizations

Consider the problem

$$\begin{aligned} \min_x \ (\max_x) \quad & x^T A x \\ \text{subject to} \quad & x^T B x = 1 \end{aligned}$$

for a matrix B symmetric and positive definite.

This can still be reduced to an eigenvlue problem in one of two ways:

1. Consider the *Generalized* eigenvalue problem

$$A\Phi = \Phi B \Lambda$$

its minimum and maximum generalized eigenvalues/eigenvectors offer the solution

Generalizations

2. Under the substitution $y = B^{1/2}$ and the problem becomes the standard Rayleigh-Ritz problem

$$\begin{aligned} \min_y \ (\max_y) \quad & y^T B^{-1/2} A B^{-1/2} y \\ \text{subject to} \quad & y^T y = 1 \end{aligned}$$

Generalizations

Consider the problem

$$\begin{aligned} \min_X \ (\max_X) \quad & \mathrm{Tr}\left(X^T A X\right) \\ \text{subject to} \quad & X^T B X = I \end{aligned}$$

for an $n \times k$ matrix X.

This can be solved takeing the k *smallest* (largest) eigenvectors of the generalized eigenvector problem, *i.e.*, k eigenvectors corresponding to the smallest (largest) eigenvalues

$$X = (\phi_1 | \phi_2 | \cdots | \phi_k)$$